Java JDBC CRUD Application

Technical Project Report

Submitted by:
Sannidhya Kumar Sonber
Reg. No: 24BAI10049

Under the Guidance of:
Dr. C. S. Rajput

VIT Bhopal University
School of Computing Science Engineering and Artificial Intelligence (SCAI)
Academic Year 2025–2026

1. Abstract

This report documents the design and implementation of a Java-based CRUD (Create, Read, Update, Delete) application that interfaces with a MySQL relational database via JDBC (Java Database Connectivity). The project demonstrates core database-connectivity concepts in Java, structured SQL execution using PreparedStatement, result set processing, and the development of two application frontends: a console-based program (Main.java) and a Java Swing graphical interface (GUI.java). The application manages user records in a MySQL database, showcasing practical skills in software development, database schema design, exception handling, and application deployment.

2. Introduction

Database-driven applications are fundamental to modern software systems. Java provides a standardised mechanism for database interaction through the JDBC API, which acts as an abstraction layer between Java code and underlying relational database systems. This project builds a functional, two-interface CRUD application targeting a MySQL database.

The project comprises two source files: Main.java, a sequential console application that demonstrates all CRUD operations programmatically, and GUI.java, a Swing-based desktop application that provides an interactive user interface for the same operations. Both files share the same database schema and JDBC connection strategy, making the project a cohesive study of Java database integration at different levels of user interaction.

3. Objectives

- Understand and apply the JDBC API for Java-to-MySQL connectivity.
- Design a normalised database schema suitable for a user management system.
- Implement all four CRUD operations using parameterised SQL queries.
- Retrieve auto-generated primary keys after INSERT operations.
- Develop a console application (Main.java) for sequential, programmatic CRUD demonstrations.
- Develop a GUI application (GUI.java) using Java Swing with interactive form fields and a live data table.
- Apply proper resource management using try-with-resources for JDBC objects.
- Handle SQL exceptions gracefully with descriptive error messages.

4. Technologies Used

Technology	Version	Purpose

Java (JDK)	8+	Core programming language and runtime
MySQL Server	8.x	Relational database management system
MySQL Connector/J	8.x	Official JDBC driver for MySQL
Java Swing	Built-in	GUI framework for the desktop interface
MySQL Workbench / CLI	Any	Database setup and administration
Terminal / CMD	OS default	Compilation and execution environment

5. System Design

5.1 Database Schema

The application uses a single MySQL database named companydb containing a users table with the following structure:

Column	Data Type	Constraints	Description
id	INT	PRIMARY KEY, AUTO_INCREMENT	Unique row identifier
name	VARCHAR(100)	NOT NULL	Full name of the user
email	VARCHAR(100)	NOT NULL, UNIQUE	Unique email address

5.2 SQL Setup Commands

The following SQL statements initialise the database environment before running the application:

```
CREATE DATABASE companydb;
USE companydb;

CREATE TABLE users (
    id    INT AUTO_INCREMENT PRIMARY KEY,
    name  VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE
);

DESCRIBE users;   -- Verify table structure
```

5.3 JDBC Connection Parameters

Both source files use the same JDBC connection constants defined at the top of their respective classes:

```
private static final String DB_URL =
    "jdbc:mysql://localhost:3306/companydb";
private static final String USER = "root";
private static final String PASS = "your_password"; // Update before running
```

The connection URL specifies the JDBC protocol (jdbc:mysql), the server host and port (localhost:3306), and the target database (companydb). A new Connection object is opened for each database operation using a try-with-resources block, ensuring automatic resource closure.

6. Implementation

6.1 Console Application – Main.java

Main.java is a sequential demonstration program that executes all four CRUD operations automatically on startup. Key design decisions include:

- Parameterised PreparedStatement for INSERT, UPDATE, and DELETE to prevent SQL injection.
- Statement.RETURN_GENERATED_KEYS flag passed to prepareStatement, enabling retrieval of the auto-incremented id after each INSERT via getGeneratedKeys().
- A reusable User POJO (Plain Old Java Object) with getters, setters, and a toString() override for clean console output.
- All JDBC resources (Connection, PreparedStatement, ResultSet) managed with try-with-resources for reliable cleanup.
- SQL exceptions propagated with throws SQLException and caught in main() with descriptive error output.

6.1.1 createUser() – INSERT with generated key retrieval

```
public static int createUser(User user) throws SQLException {
    String sql = "INSERT INTO users (name, email) VALUES (?, ?)";
    int genId = -1;
    try (Connection conn = DriverManager.getConnection(
             DB_URL, USER, PASS);
         PreparedStatement ps = conn.prepareStatement(
             sql, Statement.RETURN_GENERATED_KEYS)) {
        ps.setString(1, user.getName());
        ps.setString(2, user.getEmail());
        if (ps.executeUpdate() > 0) {
            try (ResultSet rs = ps.getGeneratedKeys()) {
                if (rs.next()) genId = rs.getInt(1);
            }
        }
    }
    return genId;
}
```

6.1.2 getAllUsers() – SELECT with ResultSet iteration

```
public static List<User> getAllUsers() throws SQLException {
    List<User> users = new ArrayList<>();
    String sql = "SELECT * FROM users";
    try (Connection conn = DriverManager.getConnection(
             DB_URL, USER, PASS);
         PreparedStatement ps = conn.prepareStatement(sql);
         ResultSet rs = ps.executeQuery()) {
        while (rs.next())
            users.add(new User(rs.getInt("id"),
                rs.getString("name"), rs.getString("email")));
    }
```

```
    return users;
}
```

6.1.3 updateUser() and deleteUser()

```
// UPDATE
public static void updateUser(int id, String name, String email)
        throws SQLException {
    String sql = "UPDATE users SET name=?, email=? WHERE id=?";
    try (Connection conn = DriverManager.getConnection(
             DB_URL, USER, PASS);
         PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, name);
        ps.setString(2, email);
        ps.setInt(3, id);
        ps.executeUpdate();
    }
}

// DELETE
public static void deleteUser(int id) throws SQLException {
    String sql = "DELETE FROM users WHERE id=?";
    try (Connection conn = DriverManager.getConnection(
             DB_URL, USER, PASS);
         PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setInt(1, id);
        ps.executeUpdate();
    }
}
```

6.2 GUI Application – GUI.java

GUI.java extends the application with a Java Swing desktop interface. It provides form inputs for user data, buttons for each CRUD operation, and a JTable that reflects the current database state in real time. Key design aspects include:

- BorderLayout with the form panel in NORTH, the JTable in CENTER, and action buttons in SOUTH.
- DefaultTableModel backing the JTable, refreshed after every database operation by calling refreshUserTable().
- A MouseListener on the JTable populates the form fields when a row is clicked, enabling update and delete by selection rather than manual ID entry.
- The ID field (idField) is set non-editable to prevent accidental modification of primary keys.
- Input validation guards against empty name or email before attempting a database operation.
- JOptionPane dialogs provide confirmation for destructive operations (delete) and feedback dialogs for success or error states.

6.2.1 Swing Layout and Table Binding

```
// Table setup
tableModel = new DefaultTableModel(new String[]{"ID","Name","Email"}, 0);
userTable  = new JTable(tableModel);
JScrollPane scrollPane = new JScrollPane(userTable);
```

```
// Row click -> populate form
userTable.addMouseListener(new MouseAdapter() {
    public void mouseClicked(MouseEvent e) {
        int row = userTable.getSelectedRow();
        if (row >= 0) {
            idField.setText(tableModel.getValueAt(row,0).toString());
            nameField.setText(tableModel.getValueAt(row,1).toString());
            emailField.setText(tableModel.getValueAt(row,2).toString());
        }
    }
});
```

6.2.2 addUser() with validation

```
private void addUser() {
    String name  = nameField.getText().trim();
    String email = emailField.getText().trim();
    if (name.isEmpty() || email.isEmpty()) {
        JOptionPane.showMessageDialog(this,
            "Name and Email cannot be empty.",
            "Input Error", JOptionPane.WARNING_MESSAGE);
        return;
    }
    try {
        createUser(new User(0, name, email));
        refreshUserTable();
        clearFields();
        JOptionPane.showMessageDialog(this,
            "User added successfully!", "Success",
            JOptionPane.INFORMATION_MESSAGE);
    } catch (SQLException e) {
        JOptionPane.showMessageDialog(this, e.getMessage(),
            "Database Error", JOptionPane.ERROR_MESSAGE);
    }
}
```

7. Full Source Code

7.1 Main.java – Console CRUD Application

```
import java.util.ArrayList;
import java.util.List;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

class User {
    private int id;
    private String name;
    private String email;

    public User(int id, String name, String email) {
        this.id = id;
        this.name = name;
        this.email = email;
    }
```

```
    public int getId()           { return id; }
    public String getName()      { return name; }
    public String getEmail()     { return email; }
    public void setName(String name)   { this.name = name; }
    public void setEmail(String email) { this.email = email; }

    @Override
    public String toString() {
        return "User{id=" + id + ", name='" + name
             + "', email='" + email + "'}";
    }
}

public class Main {

    private static final String DB_URL =
        "jdbc:mysql://localhost:3306/companydb";
    private static final String USER = "root";
    private static final String PASS = "password"; // Change this

    public static void main(String[] args) {
        try {
            System.out.println("--- Starting JDBC CRUD Demo ---");

            System.out.println("\n1. CREATING new users...");
            int id1 = createUser(new User(0, "Ada Lovelace",
                                          "ada@example.com"));
            int id2 = createUser(new User(0, "Grace Hopper",
                                          "grace@example.com"));
            System.out.println("Created IDs: " + id1 + " and " + id2);

            System.out.println("\n2. READING all users...");
            getAllUsers().forEach(System.out::println);

            System.out.println("\n3. UPDATING user ID " + id1 + "...");
            updateUser(id1, "Ada Lovelace King",
                            "ada.king@example.com");
            getAllUsers().forEach(System.out::println);

            System.out.println("\n4. DELETING user ID " + id1 + "...");
            deleteUser(id1);
            getAllUsers().forEach(System.out::println);

            System.out.println("\n--- Demo Finished ---");
        } catch (SQLException e) {
            System.err.println("DB Error: " + e.getMessage());
        }
    }

    public static int createUser(User user) throws SQLException {
        String sql = "INSERT INTO users (name, email) VALUES (?, ?)";
        int genId = -1;
        try (Connection conn = DriverManager.getConnection(
                 DB_URL, USER, PASS);
             PreparedStatement ps = conn.prepareStatement(
                 sql, Statement.RETURN_GENERATED_KEYS)) {
            ps.setString(1, user.getName());
            ps.setString(2, user.getEmail());
            if (ps.executeUpdate() > 0) {
                try (ResultSet rs = ps.getGeneratedKeys()) {
                    if (rs.next()) genId = rs.getInt(1);
                }
            }
        }
        return genId;
    }

    public static List<User> getAllUsers() throws SQLException {
        List<User> users = new ArrayList<>();
```

```
        String sql = "SELECT * FROM users";
        try (Connection conn = DriverManager.getConnection(
                 DB_URL, USER, PASS);
             PreparedStatement ps = conn.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            while (rs.next())
                users.add(new User(rs.getInt("id"),
                    rs.getString("name"), rs.getString("email")));
        }
        return users;
    }

    public static void updateUser(int id, String name,
                                  String email) throws SQLException {
        String sql = "UPDATE users SET name=?, email=? WHERE id=?";
        try (Connection conn = DriverManager.getConnection(
                 DB_URL, USER, PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, name);
            ps.setString(2, email);
            ps.setInt(3, id);
            ps.executeUpdate();
        }
    }

    public static void deleteUser(int id) throws SQLException {
        String sql = "DELETE FROM users WHERE id=?";
        try (Connection conn = DriverManager.getConnection(
                 DB_URL, USER, PASS);
             PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setInt(1, id);
            ps.executeUpdate();
        }
    }
}
```

7.2 GUI.java – Swing Desktop Application

```
import java.util.ArrayList;
import java.util.List;
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import java.sql.*;

class User {
    private int id;
    private String name;
    private String email;

    public User(int id, String name, String email) {
        this.id = id; this.name = name; this.email = email;
    }
    public int getId()       { return id; }
    public String getName()  { return name; }
    public String getEmail() { return email; }
}

public class GUI extends JFrame {

    private static final String DB_URL =
        "jdbc:mysql://localhost:3306/companydb";
    private static final String USER = "root";
    private static final String PASS = "password"; // Change this

    private final JTextField idField    = new JTextField(5);
    private final JTextField nameField  = new JTextField(20);
    private final JTextField emailField = new JTextField(20);
    private final JTable userTable;
    private final DefaultTableModel tableModel;

    public GUI() {
        setTitle("User Management System");
        setSize(600, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        // Table
        tableModel = new DefaultTableModel(
            new String[]{"ID","Name","Email"}, 0);
        userTable = new JTable(tableModel);
        JScrollPane scrollPane = new JScrollPane(userTable);

        // Form
        JPanel form = new JPanel(new GridLayout(4, 2, 5, 5));
        idField.setEditable(false);
        form.add(new JLabel("ID:"));    form.add(idField);
        form.add(new JLabel("Name:")); form.add(nameField);
        form.add(new JLabel("Email:")); form.add(emailField);

        // Buttons
        JPanel btnPanel = new JPanel();
        JButton add    = new JButton("Add User");
        JButton update = new JButton("Update Selected");
        JButton delete = new JButton("Delete Selected");
        JButton clear  = new JButton("Clear Fields");
        btnPanel.add(add); btnPanel.add(update);
        btnPanel.add(delete); btnPanel.add(clear);

        setLayout(new BorderLayout(10, 10));
```

```
        add(form, BorderLayout.NORTH);
        add(scrollPane, BorderLayout.CENTER);
        add(btnPanel, BorderLayout.SOUTH);

        add.addActionListener(e -> addUser());
        update.addActionListener(e -> updateUser());
        delete.addActionListener(e -> deleteUser());
        clear.addActionListener(e -> clearFields());

        userTable.addMouseListener(new MouseAdapter() {
            public void mouseClicked(MouseEvent e) {
                int row = userTable.getSelectedRow();
                if (row >= 0) {
                    idField.setText(
                        tableModel.getValueAt(row,0).toString());
                    nameField.setText(
                        tableModel.getValueAt(row,1).toString());
                    emailField.setText(
                        tableModel.getValueAt(row,2).toString());
                }
            }
        });
        refreshUserTable();
    }

    private void refreshUserTable() {
        try {
            tableModel.setRowCount(0);
            for (User u : getAllUsers())
                tableModel.addRow(
                    new Object[]{u.getId(),u.getName(),u.getEmail()});
        } catch (SQLException e) {
            showError("Error loading users: " + e.getMessage());
        }
    }

    private void addUser() {
        String name = nameField.getText().trim();
        String email = emailField.getText().trim();
        if (name.isEmpty() || email.isEmpty()) {
            JOptionPane.showMessageDialog(this,
                "Name and Email cannot be empty.",
                "Input Error", JOptionPane.WARNING_MESSAGE);
            return;
        }
        try {
            createUser(new User(0, name, email));
            refreshUserTable(); clearFields();
            JOptionPane.showMessageDialog(this,
                "User added successfully!","Success",
                JOptionPane.INFORMATION_MESSAGE);
        } catch (SQLException e) { showError(e.getMessage()); }
    }

    private void updateUser() {
        if (idField.getText().isEmpty()) {
            JOptionPane.showMessageDialog(this,
                "Select a user from the table first.",
                "Selection Error", JOptionPane.WARNING_MESSAGE);
            return;
        }
        try {
            updateUserInDb(Integer.parseInt(idField.getText()),
                nameField.getText(), emailField.getText());
            refreshUserTable(); clearFields();
            JOptionPane.showMessageDialog(this,
                "User updated!","Success",
                JOptionPane.INFORMATION_MESSAGE);
        } catch (SQLException e) { showError(e.getMessage()); }
    }
```

```
private void deleteUser() {
    if (idField.getText().isEmpty()) {
        JOptionPane.showMessageDialog(this,
            "Select a user from the table first.",
            "Selection Error", JOptionPane.WARNING_MESSAGE);
        return;
    }
    if (JOptionPane.showConfirmDialog(this,
            "Delete this user?","Confirm",
            JOptionPane.YES_NO_OPTION) == JOptionPane.YES_OPTION) {
        try {
            deleteUserFromDb(Integer.parseInt(idField.getText()));
            refreshUserTable(); clearFields();
        } catch (SQLException e) { showError(e.getMessage()); }
    }
}

private void clearFields() {
    idField.setText(""); nameField.setText("");
    emailField.setText(""); userTable.clearSelection();
}

private void showError(String msg) {
    JOptionPane.showMessageDialog(this, msg,
        "Database Error", JOptionPane.ERROR_MESSAGE);
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new GUI().setVisible(true));
}

// ── JDBC methods ──────────────────────────────────────────────

public static void createUser(User u) throws SQLException {
    try (Connection c = DriverManager.getConnection(
             DB_URL,USER,PASS);
         PreparedStatement ps = c.prepareStatement(
             "INSERT INTO users(name,email) VALUES(?,?)")) {
        ps.setString(1,u.getName());
        ps.setString(2,u.getEmail());
        ps.executeUpdate();
    }
}

public static List<User> getAllUsers() throws SQLException {
    List<User> list = new ArrayList<>();
    try (Connection c = DriverManager.getConnection(
             DB_URL,USER,PASS);
         Statement st = c.createStatement();
         ResultSet rs = st.executeQuery("SELECT * FROM users")) {
        while (rs.next())
            list.add(new User(rs.getInt("id"),
                rs.getString("name"),rs.getString("email")));
    }
    return list;
}

public static void updateUserInDb(int id, String name,
        String email) throws SQLException {
    try (Connection c = DriverManager.getConnection(
             DB_URL,USER,PASS);
         PreparedStatement ps = c.prepareStatement(
             "UPDATE users SET name=?,email=? WHERE id=?")) {
        ps.setString(1,name); ps.setString(2,email);
        ps.setInt(3,id); ps.executeUpdate();
    }
}
```

```
    public static void deleteUserFromDb(int id) throws SQLException {
        try (Connection c = DriverManager.getConnection(
                 DB_URL,USER,PASS);
             PreparedStatement ps = c.prepareStatement(
                 "DELETE FROM users WHERE id=?")) {
            ps.setInt(1,id); ps.executeUpdate();
        }
    }
}
```

8. Project Setup and Execution

8.1 Prerequisites

- Java Development Kit (JDK) 8 or above installed and JAVA_HOME configured.
- MySQL Server 8.x installed and running on localhost:3306.
- MySQL Connector/J JAR downloaded from https://dev.mysql.com/downloads/connector/j/
- Database companydb and table users created using the SQL commands in Section 5.2.
- DB_URL, USER, and PASS constants updated in both source files to match the local environment.

8.2 Compilation

Place the MySQL Connector JAR in the same directory as the .java files. Open a terminal in that directory and run:

Windows:

```
javac -cp ".;mysql-connector-j-8.x.x.jar" Main.java
javac -cp ".;mysql-connector-j-8.x.x.jar" GUI.java
```

macOS / Linux:

```
javac -cp ".:mysql-connector-j-8.x.x.jar" Main.java
javac -cp ".:mysql-connector-j-8.x.x.jar" GUI.java
```

8.3 Execution

Windows:

```
java -cp ".;mysql-connector-j-8.x.x.jar" Main
java -cp ".;mysql-connector-j-8.x.x.jar" GUI
```

macOS / Linux:

```
java -cp ".:mysql-connector-j-8.x.x.jar" Main
java -cp ".:mysql-connector-j-8.x.x.jar" GUI
```

9. Results and Output

Console application (Main.java): Upon execution, the program prints a step-by-step log of each CRUD operation. The output confirms successful connection, the IDs assigned to inserted records, the full list of users after each operation, confirmation of the update applied to a specific record, and the final state of the table after deletion.

```
--- Starting JDBC CRUD Demo ---

1. CREATING new users...
Created IDs: 1 and 2

2. READING all users...
User{id=1, name='Ada Lovelace', email='ada@example.com'}
User{id=2, name='Grace Hopper', email='grace@example.com'}

3. UPDATING user ID 1...
User{id=1, name='Ada Lovelace King', email='ada.king@example.com'}
User{id=2, name='Grace Hopper', email='grace@example.com'}

4. DELETING user ID 1...
User{id=2, name='Grace Hopper', email='grace@example.com'}

--- Demo Finished ---
```

GUI application (GUI.java): The Swing window opens displaying a form with ID, Name, and Email fields and four action buttons. The JTable is populated on launch by querying the database. Clicking a row auto-fills the form; changes are persisted and the table refreshes immediately after each operation. Dialogs confirm success or display database errors.

10. Conclusion

This project successfully demonstrates end-to-end Java database integration using JDBC and MySQL. Both the console and GUI implementations confirm the reliability of the CRUD operations, correct retrieval of generated keys, proper result set traversal, and safe resource management through try-with-resources.

The project reinforces foundational concepts of database-connected application development, including the JDBC connection lifecycle, parameterised query construction, exception propagation, and event-driven GUI programming with Java Swing. It serves as a solid foundation for developing multi-tier, enterprise-scale Java applications with richer data models and more complex business logic.

11. References

- Oracle Corporation. The Java Tutorials – JDBC Basics.
  https://docs.oracle.com/javase/tutorial/jdbc/

- MySQL AB. MySQL Connector/J 8.x Developer Guide. https://dev.mysql.com/doc/connector-j/
- MySQL AB. MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/
- Oracle Corporation. Java SE 8 API – javax.swing Package. https://docs.oracle.com/javase/8/docs/api/javax/swing/
- Horstmann, C. S. (2019). Core Java Volume II – Advanced Features (11th ed.). Prentice Hall.
- Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley.